

UNIVERSITÀ POLITECNICA DELLE MARCHE

FACOLTÀ DI INGEGNERIA

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea in Ingegneria Informatica e dell'Automazione



RELAZIONE D'ESAME

VideoMamba per l'Action Recognition nel retail

Candidato

Alessandro Pettinaro

Candidata

Valeria Cannone

ANNO ACCADEMICO 2024-2025

Introduzione	1
1 VideoMamba	2
1.1 Cos'è VideoMamba?	2
1.1.1 Architettura di VideoMamba	2
1.1.2 Versioni di VideoMamba	3
1.1.3 Punti di forza	4
2 Dataset utilizzati nel progetto	6
2.1 Dataset MERL Shopping	6
2.1.1 Distribuzione delle clip per classe	7
2.2 Dataset ExtraEye	8
3 Addestramento dei modelli	9
3.1 Pipeline di addestramento	9
3.2 Addestramento sul dataset MERL	10
3.2.1 Impostazione del problema e scelta dell'architettura	10
3.2.2 Scelte progettuali iniziali	10
3.2.3 Fine-tuning: profondità del backbone e regolarizzazione	11
3.2.4 Schema della pipeline di addestramento	11
3.2.5 Selezione dei modelli migliori	12
3.3 Addestramento sul dataset ExtraEye	13
3.3.1 Considerazioni sul dataset ExtraEye	13
3.3.2 Addestramento con data augmentation	13
3.3.3 Selezione dei modelli migliori	14
3.3.4 Addestramento con cross-validation	15
4 Risultati	16
4.1 Risultati su MERL Shopping Dataset	16
4.1.1 Risultati modello MERL-1	16
4.1.2 Risultati modello MERL2	18
4.2 Risultati del dataset ExtraEye	19
4.2.1 Risultati per la vista FPV con data augmentation	19
4.2.2 Risultati per la vista TPV con data augmentation	21
4.2.3 Risultati per la vista TOP con data augmentation	22

4.2.4	Risultati per la vista FPV con cross validation	23
4.2.5	Risultati per la vista TPV con cross validation	24
4.2.6	Risultati per la vista TOP con cross validation	24
4.3	Analisi comparativa tra MERL e ExtraEye	26
5	Conclusioni	27

Elenco delle figure

1.1	Architettura di VideoMamba	2
1.2	Versioni di VideoMamba	4
1.3	Summary del modello VideoMamba Middle	4
3.1	Schema del processo di addestramento MERL, con input di pesi e iperparametri	12
3.2	Schema del processo di addestramento per ogni vista del dataset ExtraEye, con input di pesi e iperparametri	14
4.1	Matrice di confusione modello MERL2	18
4.2	Matrice di confusione modello MERL2	19
4.3	Metriche di classificazione ottenute da FPV	20
4.4	Matrice di confusione della vista FPV	20
4.5	Metriche di classificazione ottenute da TPV	21
4.6	Metriche di classificazione ottenute da TPV	22
4.7	Metriche di classificazione ottenute dalla vista TOP	23
4.8	Matrice di confusione della vista TOP	23

Elenco delle tabelle

2.1	Distribuzione dei video per classe e vista nei dataset di training, validation e test.	8
3.1	Confronto tra i due modelli migliori selezionati (MERL-1 e MERL-2)	12
3.2	Modelli migliori con Data Augmentation)	15
4.1	Metriche medie ottenute dalla cross-validation per la vista FPV	24
4.2	Metriche medie ottenute dalla cross-validation per la vista TPV	24
4.3	Metriche medie ottenute dalla cross-validation per la vista FPV	25

L'elaborazione automatica dei video rappresenta una delle sfide più complesse e attuali nel campo del deep learning. La crescente disponibilità di contenuti multimediali e la necessità di estrarre informazioni significative in modo efficiente hanno spinto la ricerca verso modelli sempre più sofisticati, capaci di gestire sequenze video lunghe e variabili. In questo contesto si inserisce VideoMamba, un modello innovativo basato su State Space Models (SSM), progettato per superare i limiti delle architetture convenzionali basate su attention. VideoMamba adotta una pipeline leggera e scalabile, risultando particolarmente efficace per l'analisi di video di lunga durata. Nel presente progetto è stato utilizzato VisionMamba nella sua configurazione Middle, una componente del modello VideoMamba specificamente adattata per il riconoscimento visivo in sequenze video. L'obiettivo principale di questo lavoro è l'addestramento e la valutazione del modello **VideoMamba** per il riconoscimento di azioni in ambito retail, come l'ispezione dei prodotti o l'interazione con gli scaffali (presa o rilascio di articoli), utilizzando due dataset distinti: il **MERL Shopping Dataset** e **ExtraEye**. Il processo di sviluppo si è articolato in due fasi principali. Nella prima fase, è stato effettuato un *fine-tuning* del modello VideoMamba a partire dai pesi preaddestrati sul dataset **Kinetics-400**, con l'obiettivo di adattare il modello al dominio specifico del dataset MERL. Al termine di questa fase è stato selezionato il modello migliore, **MERL-2**, che ha successivamente costituito la base per la seconda fase. La seconda fase ha previsto un ulteriore fine-tuning sul dataset ExtraEye, riutilizzando i pesi di MERL-2 per valutare la capacità di generalizzazione e adattamento del modello a un nuovo compito: la classificazione binaria delle azioni *take* e *release*. Durante entrambe le fasi, sono state adottate diverse strategie di fine-tuning e tecniche di *cross-validation*, e le performance del modello sono state analizzate attraverso metriche quali **Loss**, **Accuracy**, **Precision**, **Recall** e **F1 Score**. Questa relazione descrive il processo di sviluppo, le scelte progettuali adottate e i risultati ottenuti, con un focus sulle tre principali viste di classificazione video: **FPV** (First-Person View), **TPV** (Third-Person View) e **TOP** (Top-View). Infine, è stato condotto un confronto tra alcune classi dei due dataset, al fine di valutare la coerenza, la robustezza e le eventuali criticità del modello in contesti diversi.

1.1 Cos'è VideoMamba?

VideoMamba è un modello all'avanguardia che adatta i State Space Models (SSM) al dominio video. I modelli SSM, offrono una complessità computazionale lineare, caratteristica che li rende ideali per la gestione di sequenze molto lunghe. VideoMamba estende questi concetti attraverso un'architettura completamente basata su SSM, senza l'uso di meccanismi di attenzione.

1.1.1 Architettura di VideoMamba

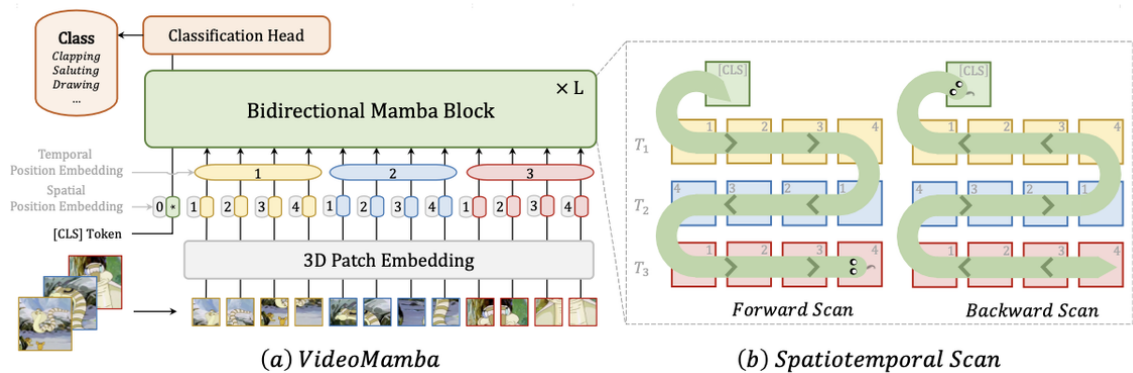


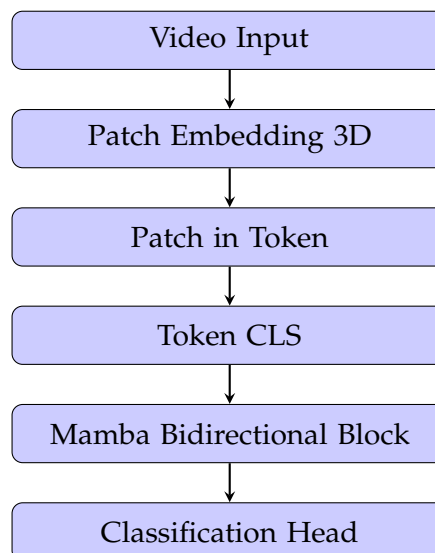
Figura 1.1: Architettura di VideoMamba

Il modello (Figura 1.1) è composto da:

- *Patch Embedding 3D*: convoluzione 3D con kernel $1 \times 16 \times 16$, che suddivide il video in patch spaziotemporali.
- *Patch $\rightarrow$ Token*: Ogni patch rappresenta una porzione locale del video, distribuita su più fotogrammi consecutivi, ed è convertita in un token. A ciascun token viene aggiunto un positional embedding, sia spaziale che temporale.

- *Token CLS*: all'inizio della sequenza viene inserito un token speciale [CLS], il cui scopo è aggregare le informazioni spaziali e temporali per fornire una rappresentazione compatta dell'intero video, utile per la classificazione finale. I token vengono inizialmente normalizzati e organizzati per posizione all'interno di ciascun fotogramma, e solo successivamente elaborati fotogramma per fotogramma lungo la sequenza temporale.
- *Blocchi Mamba Bidirezionali*: la sequenza così costruita viene elaborata da una serie di blocchi Mamba bidirezionali (B-Mamba), che operano secondo due modalità di scansione: avanti e indietro. Questo consente al modello di sfruttare appieno le conoscenze preaddestrate su immagini 2D, estendendole in modo efficiente alla dimensione temporale (3D).
- *Classification Head*: la testa di classificazione riceve i token elaborati e li utilizza per produrre la previsione finale del modello.

In maniera più schematica:



1.1.2 Versioni di VideoMamba

Il modello è disponibile in tre versioni (Figura 1.2) principali con pesi pre-addestrati di diversa dimensione per adattarsi a diversi trade-off tra accuratezza e costo computazionale. Essi sono:

- *Tiny*: leggero, rapido, ideale per dispositivi con risorse ridotte;
- *Small*: efficace bilanciamento tra prestazioni e costo;
- *Middle*: maggiore capacità e accuratezza su task complessi, con tecniche che mitigano l'overfitting.

Per il nostro addestramento, che studieremo nel Capitolo 3, abbiamo utilizzato la versione VideoMamba Middle (Figura 1.3).

Model	#Depth	#Dim	#Param.
Tiny	24	192	7M
Small	24	384	26M
Middle	32	576	74M
Base	24	768	98M

Figura 1.2: Versioni di VideoMamba

Layer (type:depth-idx)	Output Shape	Param #
VideoMambaWrapper	[1, 6]	—
└ VisionMamba: 1-1	[1, 6]	123,840
└┬ PatchEmbed: 2-1	[1, 576, 16, 14, 14]	—
└┬┬ Conv3d: 3-1	[1, 576, 16, 14, 14]	(442,944)
└┬┬ Dropout: 2-2	[1, 3137, 576]	—
└┬┬ ModuleList: 2-3	—	—
└┬┬┬ Block: 3-2	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-3	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-4	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-5	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-6	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-7	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-8	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-9	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-10	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-11	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-12	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-13	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-14	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-15	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-16	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-17	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-18	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-19	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-20	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-21	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-22	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-23	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-24	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-25	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-26	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-27	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-28	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-29	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-30	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-31	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-32	[1, 3137, 576]	(2,283,840)
└┬┬┬ Block: 3-33	[1, 3137, 576]	(2,283,840)
└┬ DropPath: 2-4	[1, 3137, 576]	—
└┬ Identity: 2-5	[1, 576]	—
└┬ Linear: 2-6	[1, 6]	(3,462)
Total params: 73,653,126		
Trainable params: 0		
Non-trainable params: 73,653,126		
Total mult-adds (G): 1.39		

Figura 1.3: Summary del modello VideoMamba Middle

1.1.3 Punti di forza

I quattro vantaggi fondamentali di VideoMamba sono:

- *Scalabilità*: il modello si adatta bene all'aumento di dimensioni del video (frame e risoluzione), mantenendo basse le richieste computazionali.
- *Sensibilità temporale*: eccelle nel riconoscimento di azioni brevi e dettagliate grazie alla possibilità di usare tecniche di masked modeling.
- *Prestazioni sui video lunghi*: supera modelli come TimeSformer e ViViT, con un consumo di memoria fino a 40× inferiore e un'inferenza 6× più veloce.

- *Compatibilità multi-modale*: si integra efficacemente in task come il video-text retrieval, superando ViT e UMT su dataset complessi e lunghi.

In sintesi, VideoMamba rappresenta un'architettura innovativa per il riconoscimento di azioni nei video, capace di combinare efficienza computazionale e prestazioni elevate grazie all'impiego dei modelli di stato (SSM) e dei blocchi Mamba bidirezionali. Dopo aver descritto la struttura e le caratteristiche principali del modello, nel capitolo successivo verrà analizzato nel dettaglio il dataset utilizzato, con particolare attenzione alla sua struttura e all'adattamento al task di classificazione video.

Dataset utilizzati nel progetto

I dataset rappresentano lo strumento fondamentale per l'addestramento di modelli di Deep Learning, poiché forniscono i dati necessari per apprendere pattern e generalizzare efficacemente su nuovi esempi.

In questo progetto sono stati utilizzati due dataset in due distinti processi di fine-tuning, condotti in sequenza:

- Il **dataset MERL Shopping** è stato utilizzato per un primo fine-tuning del modello, partendo dai pesi preaddestrati su Kinetics-400;
- Il **dataset ExtraEye** è stato successivamente impiegato per un secondo fine-tuning, utilizzando come punto di partenza i pesi ottenuti dal fine-tuning sul dataset MERL.

2.1 Dataset MERL Shopping

MERL è un dataset progettato per l'addestramento e la valutazione di algoritmi di rilevamento delle azioni. Esso è composto da 106 video in formato `.avi`, ciascuno della durata di circa 2 minuti.

I video sono stati acquisiti da una telecamera fissa posizionata dall'alto, che riprende soggetti intenti a fare acquisti all'interno di un supermercato. Il dataset è organizzato in sei classi di azioni, suddivise nei consueti insiemi di addestramento, validazione e test.

Per adattare meglio il dataset all'addestramento dei modelli, i video originali sono stati segmentati in clip di circa 10 secondi, ciascuna etichettata con l'azione contenuta. Questa operazione ha prodotto un totale di circa 5467 clip, distribuite tra le sei classi di azioni.

Le classi presenti nel dataset sono:

- *Background* (109 clip): rappresenta semplicemente l'ambiente statico del supermercato, senza che alcuna azione rilevante venga eseguita dai soggetti nel video. Serve come contesto di riferimento per distinguere le azioni reali.
- *Hand In Shelf* (566 clip): mostra un soggetto che introduce la mano all'interno dello scaffale, tipicamente per esplorare o prendere un prodotto.
- *Inspect Product* (677 clip): il soggetto osserva attentamente un prodotto preso dallo scaffale, ad esempio per leggerne l'etichetta o verificarne il prezzo.

- *Inspect Shelf* (812 clip): il soggetto esamina lo scaffale, probabilmente per confrontare diversi prodotti prima di effettuare una scelta.
- *Reach to Shelf* (1691 clip): raffigura il gesto iniziale in cui il soggetto allunga la mano verso lo scaffale per interagire con uno o più prodotti.
- *Retract From Shelf* (1612 clip): mostra il momento in cui il soggetto ritira la mano dallo scaffale, solitamente dopo aver preso o scartato un prodotto.

Un’osservazione importante riguarda la distribuzione delle classi: le azioni *Reach to Shelf* e *Retract From Shelf* risultano predominanti, mentre la classe *Background* è scarsamente rappresentata. Questo sbilanciamento può influenzare le prestazioni del modello, inducendolo a prediligere la predizione di una classe rispetto all’altra.

2.1.1 Distribuzione delle clip per classe

Il dataset segmentato che ci è stato fornito, è stato anch’esso suddiviso per ciascuna classe, con una ripartizione nei set di **train**, **val** e **test**. Di seguito si riporta il dettaglio per ogni classe:

- *Background*:
 - Train: 60 clip;
 - Validation: 18 clip;
 - Test: 28 clip.
- *Hand In Shelf*:
 - Train: 309 clip;
 - Validation: 90 clip;
 - Test: 164 clip.
- *Inspect Product*:
 - Train: 403 clip;
 - Validation: 123 clip;
 - Test: 148 clip.
- *Inspect Shelf*:
 - Train: 463 clip;
 - Validation: 142 clip;
 - Test: 204 clip.
- *Reach to Shelf*:
 - Train: 1008 clip;
 - Validation: 261 clip;
 - Test: 419 clip.
- *Retract From Shelf*:
 - Train: 950 clip;
 - Validation: 255 clip;
 - Test: 404 clip.

2.2 Dataset ExtraEye

Il dataset ExtraEye opera in scenari orientati all’obiettivo e ricchi di interazioni, come ad esempio i negozi al dettaglio. La raccolta è stata effettuata dall’Università di Macerata nell’ambito del progetto di ricerca nazionale ExtraEye, che coinvolge anche le Università di Udine e le università catanesi. In questo contesto, alcuni studenti dell’Università di Macerata hanno interpretato il ruolo di attori per la realizzazione delle riprese, in collaborazione con l’azienda Grottini Lab di Porto Recanati.

ExtraEye contiene solamente due tipologie di azioni:

- *Take*;
- *Release*.

Ogni azione è stata registrata in modo sincronizzato da tre angolazioni differenti:

- *TPV (Third-Person View)*: riprese effettuate da dietro il soggetto;
- *FPV (First-Person View)*: riprese effettuate frontalmente;
- *TOP*: riprese effettuate dall’alto.

Il dataset contiene un totale di 322 video in formato .mp4. È strutturato in cartelle denominate `ID_n`, dove `n` rappresenta l’identificativo del soggetto che compie l’azione. Al suo interno sono inoltre presenti:

- tre video completi che sono stati annotati e successivamente segmentati;
- un file Excel con le annotazioni delle azioni;
- tre cartelle separate contenenti le clip video segmentate secondo i punti di vista TPV, FPV e TOP.

A differenza del dataset MERL, il dataset View non era originariamente suddiviso in insiemi di addestramento, validazione e test. Tale suddivisione è stata creata manualmente in questo progetto, seguendo una ripartizione del 50% per il training, 25% per la validation e 25% per il test.

La Tabella 2.1 mostra la distribuzione dei video del dataset Extra Eye suddivisi per classe e punto di vista (view) nei set di training, validation e test.

Tabella 2.1: Distribuzione dei video per classe e vista nei dataset di training, validation e test.

Insieme	FPV		TOP		TPV	
	Take	Release	Take	Release	Take	Release
Train	38	38	38	38	35	35
Validation	8	8	8	8	7	7
Test	9	9	9	9	9	9

Addestramento dei modelli

L'addestramento è una fase cruciale nello sviluppo di un modello di Deep Learning, in quanto consente di apprendere dai dati i pattern utili per la classificazione video. In questo lavoro, il processo di addestramento è stato articolato in due fasi principali, corrispondenti ai due dataset utilizzati: **MERL Shopping** e **View Dataset**. Entrambi i dataset sono stati utilizzati per addestrare il modello VideoMamba, ma con obiettivi e configurazioni differenti.

3.1 Pipeline di addestramento

Tutti i processi di addestramento sono stati gestiti mediante una pipeline comune. In particolare:

- **Caricamento dei pesi preaddestrati e fine-tuning:** il modello è stato inizializzato caricando i pesi pre-addestrati all'interno del wrapper `VideoMambaWrapper` (classe creata per contenere il modello di videomamba);
- **Caricamento dei video:** Per gestire correttamente il caricamento e la preparazione dei video, è stata definita una classe dedicata al dataset. Questa classe si occupa di esplorare la struttura delle cartelle contenenti i video, dove ogni sottocartella rappresenta una classe specifica. A ciascuna classe viene assegnata un'etichetta numerica, che verrà poi utilizzata per la classificazione. Da ciascun video viene selezionato un numero fisso di frame, che vengono poi ridimensionati alla risoluzione richiesta dal modello, ovvero 224×224 pixel. Infine i frame vengono convertiti in tensori.
- **Configurazione dei DataLoader:** per facilitare l'iterazione sono stati creati due `DataLoader`, uno con `shuffle=True` per il training e uno con `shuffle=False` per la validazione
- **Ciclo di addestramento:** l'addestramento del modello è stato condotto per un numero fisso di epoche, stabilito in base al dataset utilizzato. Durante ogni epoca, le metriche di performance sono state monitorate e registrate tramite `TensorBoard`, così da consentire una valutazione continua dell'andamento del training. Inoltre, è stato adottato un meccanismo di `early stopping`, che interrompe anticipatamente l'addestramento nel caso in cui le prestazioni sul set di validazione non mostrino miglioramenti per un certo numero consecutivo di epoche. Il modello con la migliore accuratezza ottenuta durante l'addestramento è stato salvato automaticamente.

- **Salvataggio del modello finale:** al termine delle epoche prefissate vengono salvati i pesi del modello finale.

3.2 Addestramento sul dataset MERL

L'addestramento del modello ha richiesto una serie di scelte progettuali volte ad adattare un'architettura pre-addestrata a un nuovo compito di classificazione video. In particolare, si è trattato di definire:

- la configurazione ottimale del modello in termini di struttura e input temporali
- la profondità del fine-tuning,
- principali iperparametri di regolarizzazione.

Nel complesso, l'obiettivo è stato quello di ottenere un modello in grado di apprendere efficacemente le sei classi di azioni presenti nel dataset MERL, partendo da una solida base pre-addestrata su un ampio dataset come Kinetics-400.

3.2.1 Impostazione del problema e scelta dell'architettura

Il dataset MERL pone un problema di classificazione video multi-classe, articolato in sei categorie di azioni riprese da una prospettiva fissa. Per affrontare questo task, è stato adottato il modello VideoMamba, inizializzato con pesi pre-addestrati sul dataset Kinetics-400, una raccolta di oltre 300.000 videoclip annotati, suddivisi in 400 categorie di azioni umane (come "camminare", "correre", "aprire una porta"), provenienti da contenuti reali di YouTube. Questo pretraining ha fornito al modello una solida capacità di rappresentazione delle dinamiche temporali e dei movimenti, utile come punto di partenza per il nuovo compito. Per adattarlo al dataset MERL, è stata sostituita la classification head originale con una nuova head composta da sei unità di output, corrispondenti alle sei classi specifiche del problema.

3.2.2 Scelte progettuali iniziali

Una delle prime decisioni progettuali ha riguardato la scelta del numero di frame per clip da utilizzare in input al modello. Tale parametro influenza direttamente la quantità di informazione temporale catturata durante l'addestramento, con implicazioni sia sulle performance del modello che sul costo computazionale. Poiché VideoMamba mette a disposizione pesi pre-addestrati su Kinetics-400 per tre configurazioni standard (8, 16 e 32 frame), si è deciso di testarle tutte al fine di individuare la più adatta al dataset MERL. L'analisi ha evidenziato che:

- la configurazione a 8 frame non permetteva di apprendere efficacemente la dinamica delle azioni, portando a prestazioni inferiori;
- quella a 32 frame, sebbene più ricca dal punto di vista informativo, tendeva all'overfitting, con un peggioramento della generalizzazione;
- la configurazione a 16 frame per clip (`videomamba_m16_k400_mask_ft_f16_res224`) rappresentava il miglior compromesso tra accuratezza e rappresentazione temporale.

Sulla base di queste evidenze, è stata scelta la configurazione a 16 frame adottata per tutte le successive fasi di addestramento e fine-tuning.

3.2.3 Fine-tuning: profondità del backbone e regolarizzazione

Dopo aver selezionato la configurazione ottimale in termini di frame per clip (16 frame), è stata avviata la fase di fine-tuning, con l'obiettivo di adattare il modello pre-addestrato al dataset MERL, migliorandone la capacità di riconoscere le sei classi specifiche senza compromettere l'informazione appresa durante il pretraining su Kinetics-400.

Uno degli aspetti centrali di questa fase ha riguardato la profondità dell'aggiornamento del backbone, ovvero la scelta di quali blocchi rendere effettivamente addestrabili. Sbloccare un numero maggiore di blocchi consente una specializzazione più marcata sul nuovo compito, ma può aumentare il rischio di overfitting.

Per valutare questo bilanciamento, sono state testate tre configurazioni:

- **8 blocchi sbloccati**: configurazione conservativa, con minore capacità di adattamento;
- **10 blocchi sbloccati**: configurazione intermedia, potenzialmente più bilanciata;
- **12 blocchi sbloccati**: maggiore profondità, ma con rischio di overfitting.

I risultati hanno evidenziato che:

- la configurazione con **10 blocchi** offriva le migliori prestazioni complessive, combinando buona accuratezza e stabilità del training;
- con **8 blocchi**, il modello risultava troppo rigido e poco adattabile;
- con **12 blocchi**, si manifestavano fenomeni di overfitting.

Parallelamente, è stata condotta un'analisi degli iperparametri di regolarizzazione, con particolare attenzione a:

- **Dropout**: testati i valori 0.1 e 0.2;
- **Drop Path** : testato il valore 0.4 e 0.5.

Anche in questo caso, l'obiettivo era contenere l'overfitting senza compromettere la capacità del modello di apprendere i pattern temporali. I risultati hanno mostrato che:

- valori troppo bassi erano poco efficaci nel contrastare l'overfitting;
- valori troppo alti ostacolavano l'apprendimento dei pattern rilevanti e la convergenza del modello.

3.2.4 Schema della pipeline di addestramento

Per fornire una visione d'insieme del processo implementato, la Figura 3.2 rappresenta in modo schematico la pipeline di addestramento utilizzata per il modello VideoMamba sul dataset MERL. Il flusso ha inizio con il dataset MERL, che viene preprocessato attraverso la classe `MerlDataset`, responsabile della trasformazione dei frame in tensori. I dati vengono poi caricati in batch tramite un `DataLoader` usati durante l'addestramento, successivamente, viene utilizzata la classe `VideoMambaWrapper`, che integra il modello Video Mamba e gestisce anche la logica di sblocco dei blocchi del backbone. In parallelo, vengono caricati i pesi pre-addestrati e vengono definiti gli iperparametri di addestramento (learning rate, dropout, ottimizzatore, ecc.). Una volta completata l'inizializzazione, si procede con l'addestramento vero e proprio, al termine del quale viene salvato il modello risultante.

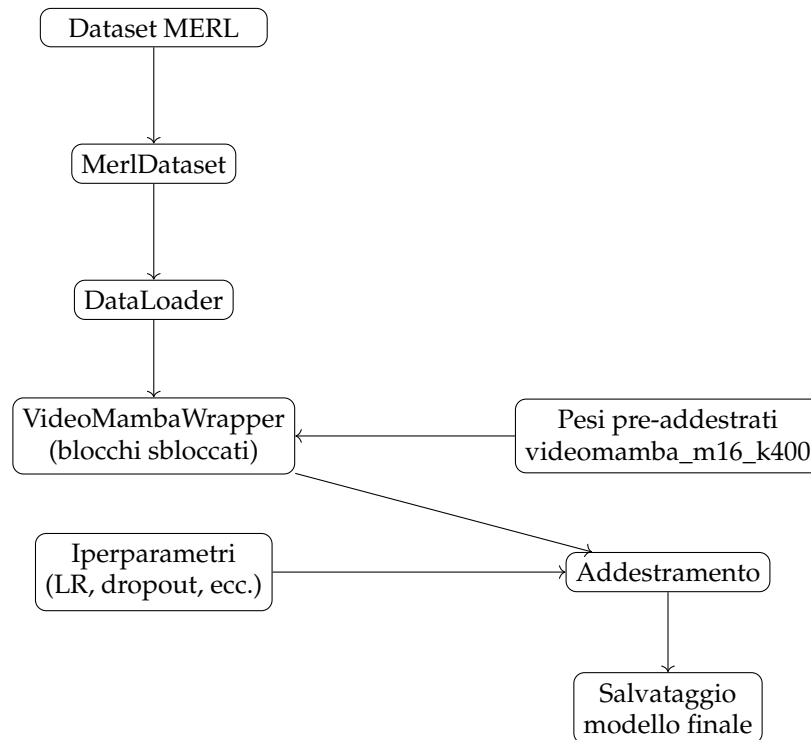


Figura 3.1: Schema del processo di addestramento MERL, con input di pesi e iperparametri

3.2.5 Selezione dei modelli migliori

Sulla base delle sperimentazioni effettuate, sono state individuate due configurazioni particolarmente valide, denominate **MERL-1** e **MERL-2**. Entrambi i modelli condividono la stessa struttura di base: utilizzano 16 frame per clip e prevedono lo sblocco di 10 blocchi del backbone durante il fine-tuning. La differenza principale tra le due configurazioni risiede nell'impiego del **weight decay**. In particolare, il modello **MERL-2** adotta questa tecnica, che si è rivelata più efficace nel limitare l'overfitting e nel garantire una maggiore stabilità dell'addestramento.

La Tabella 3.1 sintetizza le caratteristiche principali dei due modelli e le relative osservazioni emerse in fase di validazione.

Tabella 3.1: Confronto tra i due modelli migliori selezionati (MERL-1 e MERL-2)

Parametro	MERL-1	MERL-2 (scelto)
Frame per clip	16	16
Layer sbloccati	10	10
Epoche	15	35
Dropout rate	0.1	0.2
Drop path rate	0.4	0.4
Ottimizzatore	Adam	AdamW
Weight decay	No	0.005
Learning rate	0.0001	0.0002
Batch size	8	8
Stabilità del training	Alta	Alta
Generalizzazione	Buona	Buona

L'andamento dell'addestramento e i risultati ottenuti per entrambe le configurazioni sono

riportati nel Capitolo 4, dove vengono analizzate le metriche e i grafici relativi all'accuracy e alla loss durante le varie fasi di training e di validation. Sulla base di tali risultati, è stato selezionato il modello **MERL-2** per il fine-tuning sul dataset **ExtraEye**, in quanto ha dimostrato una maggiore stabilità e capacità di generalizzazione rispetto a MERL-1.

3.3 Addestramento sul dataset ExtraEye

Dopo l'addestramento e l'ottimizzazione del modello VideoMamba sul dataset MERL, l'obiettivo di questa fase è stato quello di riutilizzare i pesi appresi durante il primo addestramento per affrontare un nuovo compito: la classificazione binaria delle azioni *take* e *release* all'interno del dataset **ExtraEye**. A tal fine, il modello è stato inizializzato con i pesi del modello MERL-2 e adattato al nuovo task mediante la modifica della classification head, ridotta a due unità di output. Questa strategia risulta particolarmente efficace considerando che alcune azioni presenti in MERL, come ad esempio *inspection product*, mostrano dinamiche visive simili all'azione *take* del nuovo task.

Il dataset utilizzato per questa fase, denominato ExtraEye, è composto da registrazioni video di azioni umane raccolte da tre diverse prospettive di osservazione:

- **FPV** (First-Person View), ovvero la vista soggettiva dell'agente che compie l'azione;
- **TPV** (Third-Person View), ossia una vista esterna da dietro le spalle del soggetto;
- **TOP**, ovvero una ripresa dall'alto.

3.3.1 Considerazioni sul dataset ExtraEye

L'addestramento di un modello di deep learning su dataset di dimensioni ridotte comporta numerose sfide, in particolare in termini di generalizzazione e rischio di overfitting. Il dataset ExtraEye, sebbene molto interessante dal punto di vista applicativo, presenta alcune caratteristiche che ne rendono l'utilizzo particolarmente delicato in fase di training. In particolare, il numero di campioni a disposizione è sensibilmente inferiore rispetto al dataset MERL, rendendo più complesso per il modello apprendere.

Per affrontare questa difficoltà e garantire un addestramento efficace e il più possibile generalizzabile, sono state adottate due strategie distinte:

1. un addestramento con *data augmentation*, finalizzata ad aumentare artificialmente la variabilità del training set;
2. un addestramento basata su *cross-validation*.

3.3.2 Addestramento con data augmentation

Dato che il dataset ExtraEye presenta una quantità molto limitata di campioni, è stato necessario intervenire per migliorare la capacità del modello di generalizzare su dati non visti. Per questo motivo, è stata introdotta una procedura di espansione controllata del training set attraverso l'applicazione di tecniche di *data augmentation*.

L'augmentazione è stata progettata con l'obiettivo di simulare realistiche variazioni visive senza alterare la struttura semantica delle sequenze. È stato scelto di non applicare trasformazioni geometriche come rotazioni o riflessioni, poiché gli scaffali presenti nei video sono posizionati in un punto fisso dell'inquadratura.

L'aumento dei dati è stato quindi realizzato tramite modifiche cromatiche leggere dei frame, applicate con probabilità del 50% su ciascun video. In particolare, ogni video aveva

al massimo una copia aumentata. Le trasformazioni sono state definite mediante la funzione `ColorJitter` della libreria `torchvision.transforms`, configurata con i seguenti parametri:

- **Luminosità:** variazione fino al 10%;
- **Contrasto:** variazione fino al 50%;
- **Saturazione:** variazione fino al 50%;
- **Tonalità:** variazione massima di 0.2.

Queste modifiche hanno permesso di introdurre una maggiore diversità visiva nei dati di addestramento, pur mantenendo intatta la coerenza spaziale e semantica delle scene.

Come già avvenuto per il dataset MERL, sono state inoltre testate diverse configurazioni del modello, variando il numero di blocchi sbloccati (2, 4 e 6) e i valori di *Dropout* e *Drop Path*.

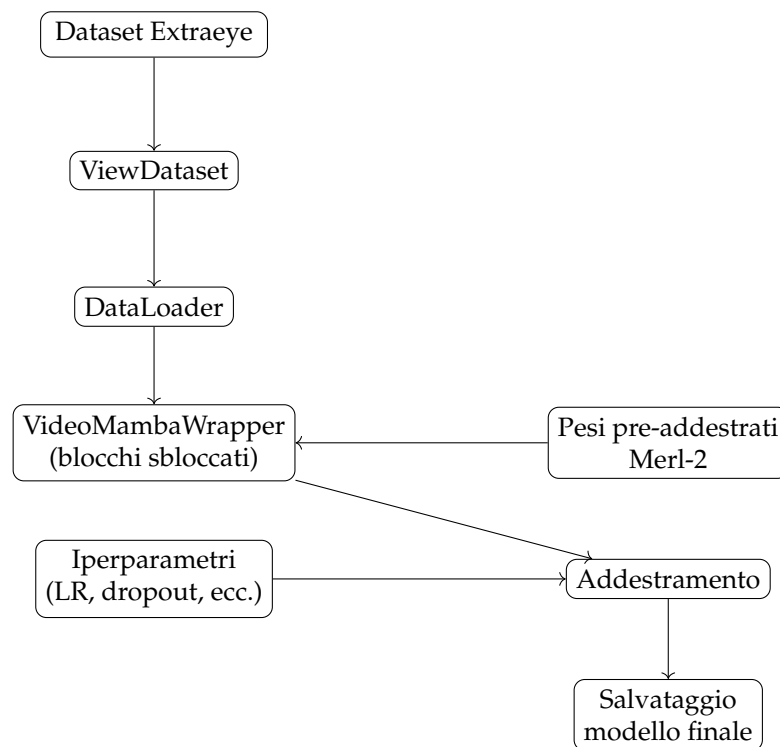


Figura 3.2: Schema del processo di addestramento per ogni vista del dataset ExtraEye, con input di pesi e iperparametri

3.3.3 Selezione dei modelli migliori

Dopo una serie di sperimentazioni condotte separatamente per ciascuna delle tre viste (FPV, TPV e TOP), è stato possibile identificare una configurazione di iperparametri comune che garantisse un buon compromesso tra stabilità del training e capacità di generalizzazione. La Tabella 3.2 riassume i parametri adottati nella configurazione finale, selezionata al termine della fase con *data augmentation*.

Parametro	FPV-TPV-TOP
Frame per clip	16
Layer sbloccati	4
Epoche	15
Dropout rate	0.1
Drop path rate	0.4
Ottimizzatore	Adam
Weight decay	No
Learning rate	0.0001
Batch size	8
Stabilità del training	Normale
Generalizzazione	Normale

Tabella 3.2: Modelli migliori con Data Augmentation)

3.3.4 Addestramento con cross-validation

Come discusso in precedenza, il dataset ExtraEye presenta una quantità limitata di campioni, il che rende particolarmente difficile ottenere un modello generalizzabile e robusto. Per affrontare tale criticità, è stata adottata una seconda strategia di addestramento basata sulla cross-validation. La cross-validation è una tecnica utilizzata nell'apprendimento automatico per valutare l'affidabilità e la capacità di generalizzazione di un modello, riducendo al contempo il rischio di overfitting. A differenza della classica suddivisione dei dati in un unico training set, validation set e test set, la cross-validation prevede una suddivisione del dataset in più sottoinsiemi (o fold), sui quali si eseguono iterativamente fasi di addestramento e validazione. In ogni iterazione, il modello viene addestrato su una combinazione di fold e validato sui rimanenti. In questo progetto, la procedura è stata implementata tramite uno script in Python che gestisce in modo automatico la generazione dei vari fold, strutturati in file .json, e l'avvio ciclico delle sessioni di training e validazione per ciascun fold. La strategia di suddivisione adottata è stata pensata appositamente per il dataset ExtraEye, che include 17 soggetti distinti, ciascuno con i propri video. L'obiettivo è valutare la capacità del modello di generalizzare su soggetti non visti in fase di addestramento.

La logica di costruzione dei fold adottata è la seguente:

- Ad ogni iterazione (*fold*):
 - si seleziona un **soggetto** come **test set**;
 - si seleziona un secondo **soggetto**, diverso dal precedente, come **validation set**;
 - i **15 soggetti rimanenti** vengono utilizzati per il **training set**.
- Il processo viene ripetuto ciclicamente per tutte le possibili combinazioni *test/validation*.
- In questo modo, ogni soggetto viene utilizzato almeno una volta in ciascun ruolo (test, validation, training).

In questo capitolo vengono presentati i risultati migliori ottenuti dagli addestramenti effettuati sul dataset MERL e sulle tre viste del dataset ExtraEye. Verranno inoltre illustrati i risultati delle configurazioni con cross-validation applicate al dataset ExtraEye, al fine di fornire una valutazione più robusta delle prestazioni del modello e notare gli effetti distintivi di questa tecnica.

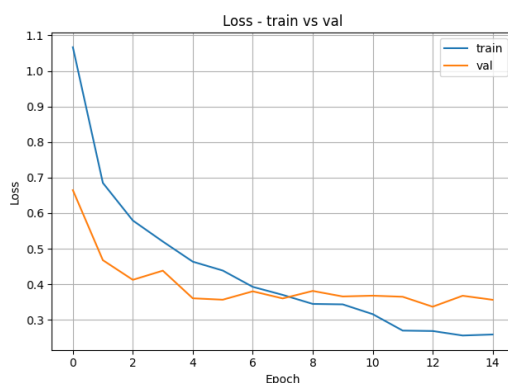
4.1 Risultati su MERL Shopping Dataset

In questa sezione sono illustrati e discussi i risultati degli addestramenti effettuati sul dataset MERL.

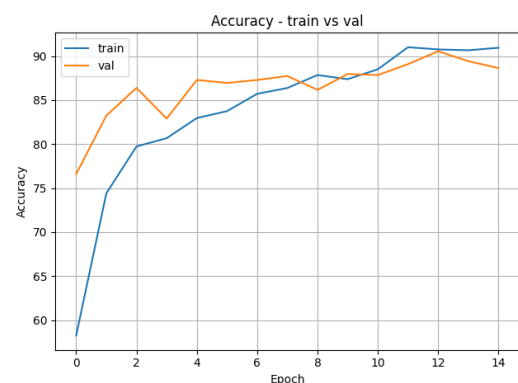
4.1.1 Risultati modello MERL-1

Per una valutazione completa delle prestazioni del modello MERL-1, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello.

Andamento durante l'addestramento



(a) Loss durante l'addestramento



(b) Accuracy durante l'addestramento

Test

	precision	recall	f1-score
Background	1.00	1.00	1.00
Hand In Shelf	0.85	0.74	0.79
Inspect Product	0.65	0.80	0.72
Inspect Shelf	0.88	0.82	0.85
Reach To Shelf	0.93	0.90	0.92
Retract From Shelf	0.90	0.92	0.91
accuracy	0.87	0.87	0.87
macro avg	0.87	0.87	0.86
weighted avg	0.87	0.87	0.87

(a) Metriche di classificazione

True Label \ Predicted Label	Background	Hand In Shelf	Inspect Product	Inspect Shelf	Reach To Shelf	Retract From Shelf
Background	28	0	0	0	0	0
Hand In Shelf	0	122	8	0	17	17
Inspect Product	0	2	119	11	5	11
Inspect Shelf	0	1	23	167	4	9
Reach To Shelf	0	15	15	5	379	5
Retract From Shelf	0	4	18	7	2	373

(b) Matrice di confusione

Osservazioni

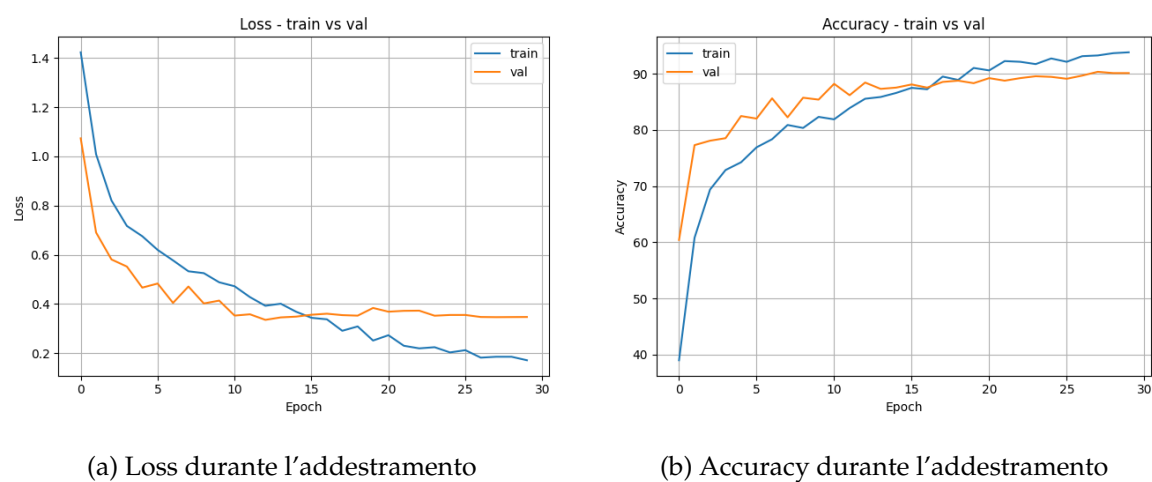
Il modello MERL-1 ha mostrato un comportamento di apprendimento stabile durante l'addestramento, con curve di loss e accuracy regolari. Le prestazioni sul test set sono risultate complessivamente soddisfacenti: il modello ha riconosciuto correttamente la maggior parte delle azioni, in particolare Reach To Shelf (379 predizioni corrette) e Retract From Shelf (373), con un numero minimo di errori. Anche le classi Inspect Shelf (167) e Inspect Product (119) sono state gestite con buona precisione, sebbene si sia osservata una certa confusione tra queste due categorie, verosimilmente a causa della loro somiglianza visiva. Un fenomeno simile è emerso anche tra le azioni Hand In Shelf e Retract From Shelf, che presentano movimenti

parzialmente sovrapposti. La classe Background è stata sempre distinta correttamente (28 su 28). Le metriche di valutazione confermano questi risultati: la maggior parte delle classi presenta precision e F1-score elevati, con l'unica eccezione di Inspect Product, che registra una precision pari a 0.65 e un F1-score di 0.72.

4.1.2 Risultati modello MERL2

Per una valutazione completa delle prestazioni del modello MERL-2, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello.

Andamento durante l'addestramento



Test

Classification Report				
Class	Background	0.97	1.00	0.98
	Hand In Shelf	0.77	0.84	0.80
	Inspect Product	0.73	0.72	0.72
	Inspect Shelf	0.87	0.86	0.86
	Reach To Shelf	0.92	0.91	0.92
	Retract From Shelf	0.94	0.92	0.93
	accuracy	0.88	0.88	0.88
	macro avg	0.86	0.88	0.87
	weighted avg	0.88	0.88	0.88
	precision	recall	f1-score	
	Metric			

Figura 4.1: Matrice di confusione modello MERL2

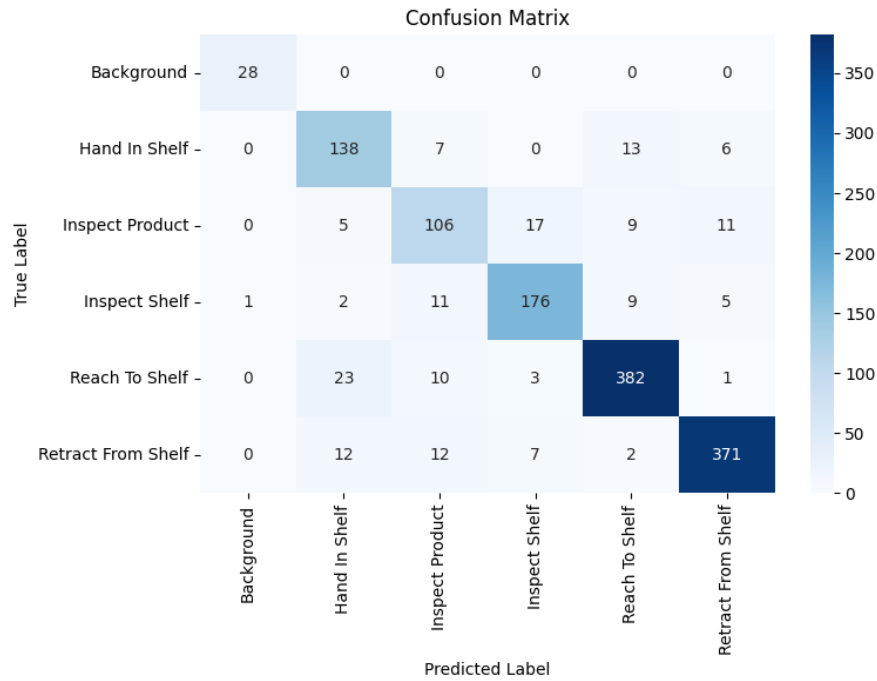


Figura 4.2: Matrice di confusione modello MERL2

Osservazioni

Il modello MERL-2 ha mostrato un apprendimento stabile e continuo, senza segni di overfitting anche dopo molte epoche. Le migliori performance sono state ottenute per le classi Reach To Shelf (377) e Retract From Shelf (375), mentre alcune confusioni sono emerse tra azioni visivamente simili come Inspect Product, Inspect Shelf e Hand In Shelf. Background è sempre stato riconosciuto correttamente. Le metriche confermano precision e F1-score elevati per quasi tutte le classi, con difficoltà localizzate soprattutto su Inspect Product e Hand in shelf, a causa della minore presenza di video di addestramento per questa classe. Dalla matrice di confusione e dal report di classificazione (Figura 4.4), possiamo osservare che il modello MERL2 ottiene prestazioni migliori rispetto al modello MERL1. Per questo motivo, è stato selezionato per eseguire il fine-tuning sul dataset ExtraEye.

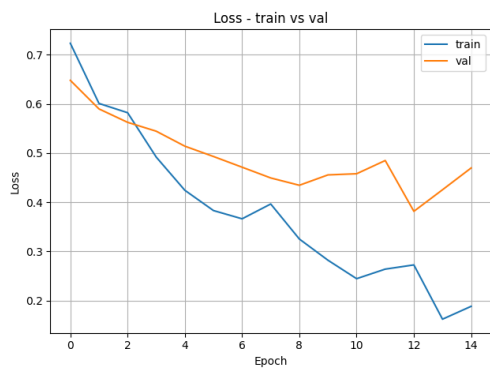
4.2 Risultati del dataset ExtraEye

In questa sezione sono illustrati i risultati ottenuti sul dataset ExtraEye, addestrato utilizzando i pesi del modello MERL2. Sono state applicate due tecniche: data augmentation e cross-validation. I risultati relativi a entrambe le modalità di addestramento sono riportati e analizzati di seguito.

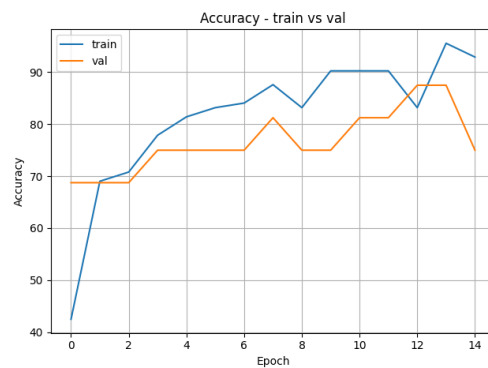
4.2.1 Risultati per la vista FPV con data augmentation

Per una valutazione completa delle prestazioni del modello FPV_AG, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello.

Andamento Addestramento



(a) Loss durante l'addestramento



(b) Accuracy durante l'addestramento

Test

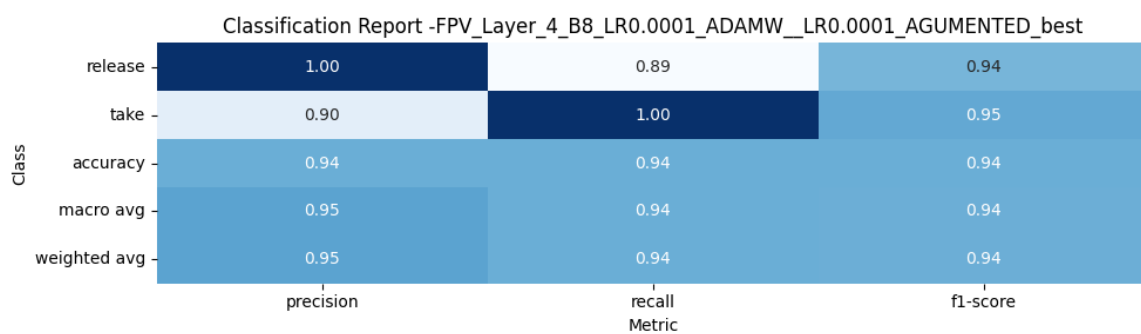


Figura 4.3: Metriche di classificazione ottenute da FPV

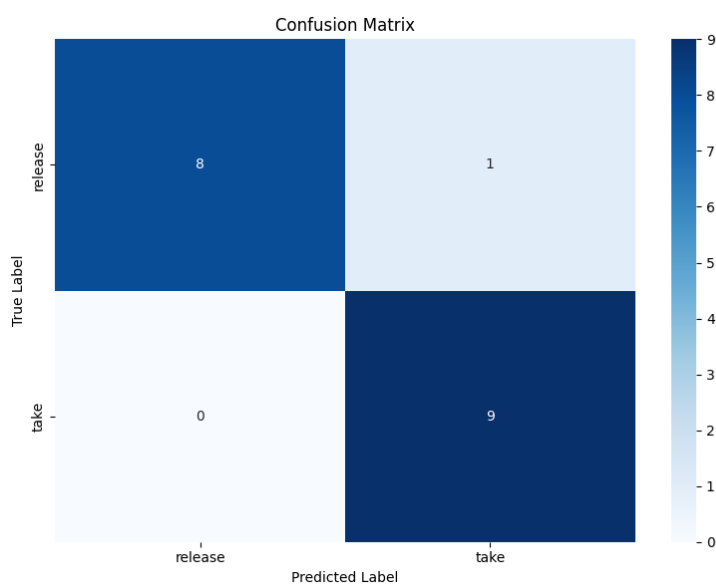


Figura 4.4: Matrice di confusione della vista FPV

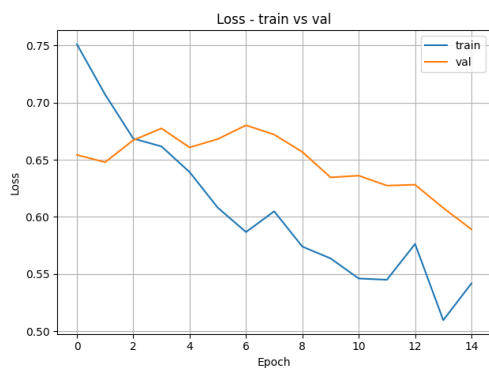
Osservazioni

Per la vista FPV_AG, l'andamento della loss è stato generalmente stabile durante l'addestramento, ma si osserva un aumento della loss di validazione nelle ultime epoche, segno di un possibile inizio di overfitting, sebbene non significativo. L'accuracy si è mantenuta alta e costante, confermando una buona capacità del modello di apprendere le dinamiche della vista. Il modello ha ottenuto ottime prestazioni, commettendo un solo errore su 18 campioni nel test set. In particolare, ha classificato correttamente tutti gli esempi di classe take e release, con un solo caso in cui un release è stato erroneamente etichettato come take. Le metriche di Precision, Recall e F1-Score risultano elevate e prossime al 100%, come evidenziato nel report di classificazione

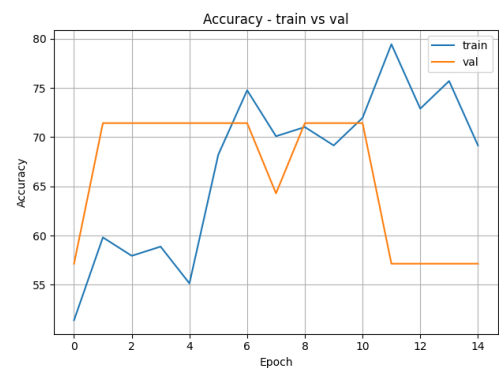
4.2.2 Risultati per la vista TPV con data augmentation

Per una valutazione completa delle prestazioni del modello TPV_AG, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello.

Andamento Addestramento



(a) Loss durante l'addestramento



(b) Accuracy durante l'addestramento

Test

Classification Report -TPV_Layer_4_B8_LR0.0001_ADAMW__best				
Class	release	0.86	0.67	0.75
	take	0.73	0.89	0.80
	accuracy	0.78	0.78	0.78
	macro avg	0.79	0.78	0.78
	weighted avg	0.79	0.78	0.77
		precision	recall	f1-score

Figura 4.5: Metriche di classificazione ottenute da TPV

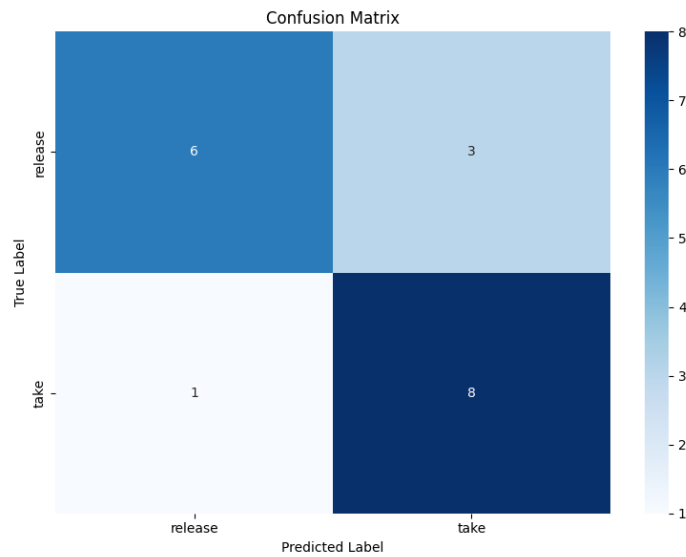


Figura 4.6: Metriche di classificazione ottenute da TPV

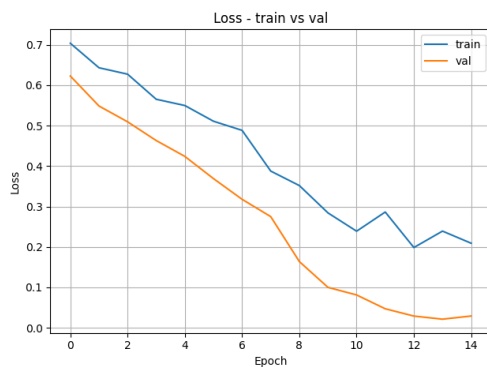
Osservazioni

Per la vista TPV_AG, la loss mostra una tendenza alla discesa sia per il training sia per la validazione, ma non raggiunge un valore ottimale, suggerendo che il modello stava ancora apprendendo e avrebbe potuto migliorare con un numero maggiore di epoche. Le prestazioni complessive sono buone: il modello ha classificato correttamente la maggior parte dei campioni, con 6 true negative, 8 true positive, 1 false negative e 3 false positive. Gli errori si concentrano principalmente nel riconoscimento della classe release, che risulta più debole.

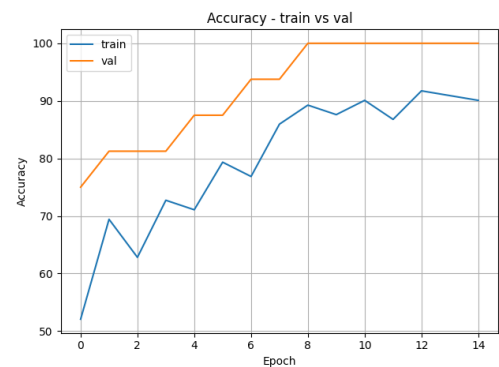
4.2.3 Risultati per la vista TOP con data augmentation

Per una valutazione completa delle prestazioni del modello TOP_AG, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello..

Andamento Addestramento



(a) Loss durante l'addestramento



(b) Accuracy durante l'addestramento

Test

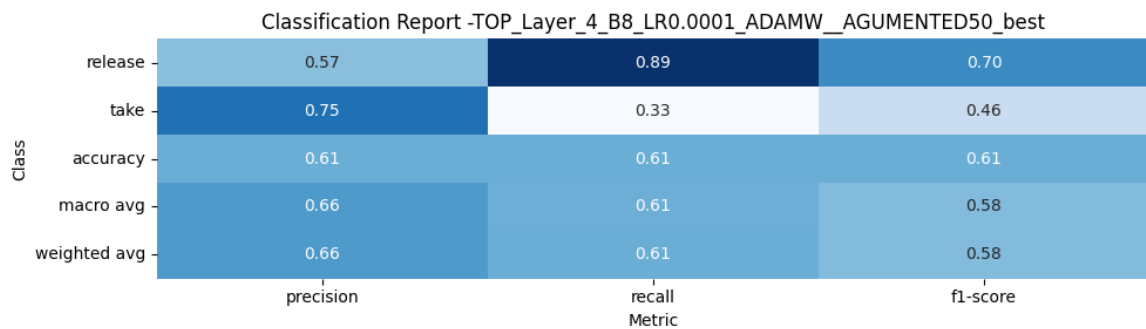


Figura 4.7: Metriche di classificazione ottenute dalla vista TOP

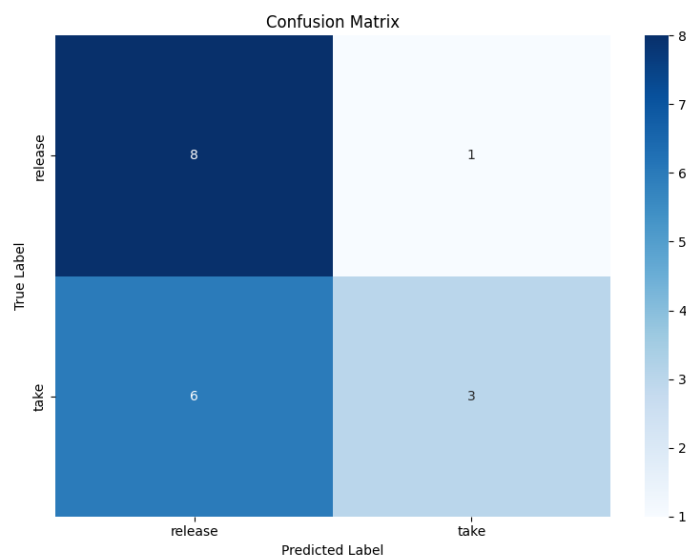


Figura 4.8: Matrice di confusione della vista TOP

Osservazioni

Per la vista TOP_AG, la loss di training raggiunge un valore ottimale, mentre la loss di validazione risulta insolitamente bassa, probabilmente a causa della quantità limitata di dati. Il modello mostra buone prestazioni sulla classe release, ma notevoli difficoltà nel riconoscere la classe take, con 6 falsi negativi su 9 campioni. Come confermato dal report di classificazione (Figura 4.10), il recall per take è solo del 33%, indicando la necessità di ulteriori ottimizzazioni, soprattutto per migliorare la sensibilità verso classi meno rappresentate.

4.2.4 Risultati per la vista FPV con cross validation

Per una valutazione completa delle prestazioni del modello FPV_CV, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello. Poiché è stata adottata una strategia di cross-validation, i risultati riportati corrispondono alla media delle metriche ottenute sui diversi fold, calcolate all'epoca in cui l'Accuracy di validazione ha raggiunto il valore massimo.

Tabella 4.1: Metriche medie ottenute dalla cross-validation per la vista FPV

Metriche	FPV_CV		
	Train	Validation	Test
Loss (%)	27.2	21.1	–
Accuracy (%)	93.22	87.67	86.5

Osservazioni

Il modello FPV_CV ha mostrato buone prestazioni stabili nei diversi fold, con metriche coerenti senza evidenti segni di overfitting. Rispetto alla versione con data augmentation (FPV_AG), il modello FPV_CV risulta più stabile, ma l'accuratezza sul test set è leggermente inferiore. FPV_AG ottiene risultati migliori in termini di accuratezza complessiva.

4.2.5 Risultati per la vista TPV con cross validation

Per una valutazione completa delle prestazioni del modello TPV_CV, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello. Poiché è stata adottata una strategia di cross-validation, i risultati riportati corrispondono alla media delle metriche ottenute sui diversi fold, calcolate all'epoca in cui l'Accuracy di validazione ha raggiunto il valore massimo.

Tabella 4.2: Metriche medie ottenute dalla cross-validation per la vista TPV

Metriche	TPV_CV		
	Train	Validation	Test
Loss (%)	58.2	55.3	–
Accuracy (%)	66.7	70	65

Osservazioni

Per la vista TPV_CV si riscontra una buona coerenza tra le metriche, con un'accuratezza stabile e bilanciata tra validazione e test. La loss media risulta tuttavia elevata, indicando margini di miglioramento. Anche il modello TPV_AG presenta una loss elevata e un apprendimento non ancora completamente consolidato, ma raggiunge comunque prestazioni soddisfacenti. Il confronto mostra una leggera superiorità in termini di accuratezza per il modello TPV_AG.

4.2.6 Risultati per la vista TOP con cross validation

Per una valutazione completa delle prestazioni del modello TOP_CV, nei paragrafi seguenti vengono presentati l'andamento delle metriche durante la fase di addestramento, i risultati finali ottenuti sul test set e alcune considerazioni qualitative sull'efficacia e la stabilità del modello. Poiché è stata adottata una strategia di cross-validation, i risultati riportati corrispondono alla media delle metriche ottenute sui diversi fold, calcolate all'epoca in cui l'Accuracy di validazione ha raggiunto il valore massimo.

Tabella 4.3: Metriche medie ottenute dalla cross-validation per la vista FPV

Metriche	TOP_CV		
	Train	Validation	Test
Loss (%)	26.9	12	–
Accuracy (%)	89.3	96.7	85.7

Osservazioni

Il modello TOP_CV mostra metriche elevate, in particolare sulla validazione, con una loss contenuta e un'accuratezza superiore al 95%; inoltre, anche l'accuratezza sul test set risulta buona. Tuttavia, rispetto all'addestramento con data augmentation (TOP_AG), il modello TOP_CV ottiene un'accuratezza inferiore sul test set, risultando meno efficace in fase di valutazione finale.

In conclusione, il modello addestrato sul dataset ExtraEye con tecniche di cross-validation ha ottenuto risultati mediamente discreti. Tuttavia, confrontando le medie delle metriche di performance, il modello basato su data augmentation ha mostrato prestazioni superiori.

4.3 Analisi comparativa tra MERL e ExtraEye

Confrontando le classi di azioni presenti nel dataset MERL e nel dataset ExtraEye, abbiamo riscontrato comportamenti analoghi tra alcune categorie.

In particolare, abbiamo osservato una forte somiglianza tra la classe *Inspect Product* del dataset MERL e la classe *take* del dataset ExtraEye. In entrambi i casi, i soggetti eseguono movimenti molto simili, come l'avvicinamento a un oggetto e la sua manipolazione, pur trovandosi in ambienti differenti ma visivamente simili.

Analizzando i risultati ottenuti, emerge una leggera criticità nella classificazione della classe *Inspect Product* all'interno del dataset MERL. In particolare, sia il modello MERL1 che MERL2 mostrano valori inferiori di precision e F1-score per questa classe rispetto alle altre, con un F1 pari a 0.72 per MERL1. Ciò suggerisce una maggiore incertezza del modello nel distinguere correttamente questa azione rispetto alle altre presenti nel dataset.

Allo stesso tempo, nei test effettuati sulla classe *take* del dataset ExtraEye, soprattutto nella vista FPV, si osservano prestazioni eccellenti, con valori di precision, recall e F1-score vicini o pari al 100%, sia con data augmentation che con cross-validation. Questo indica che, se il contesto visivo è sufficientemente omogeneo e il numero di classi è limitato, il modello riesce a distinguere molto chiaramente l'azione in analisi.

Considerando la forte somiglianza gestuale tra *Inspect Product* e *take*, è lecito pensare che, nel caso in cui i due dataset venissero utilizzati insieme nello stesso processo di addestramento, questo potrebbe generare ambiguità durante la fase di classificazione. In tal caso, il modello potrebbe incontrare difficoltà nel distinguere correttamente le due classi, aumentando il rischio di false predizioni dovute alla similarità sia visiva che gestuale delle azioni.

Il presente progetto ha avuto come obiettivo la sperimentazione del modello VideoMamba per il riconoscimento di azioni in ambito retail, utilizzando due dataset differenti: MERL Shopping Dataset e ExtraEye.

La pipeline di addestramento è stata articolata in due fasi principali: una prima fase di fine-tuning sul dataset MERL e una seconda fase di fine-tuning delle conoscenze acquisite dal precedente dataset sul dataset ExtraEye, con lo scopo di valutare la generalizzazione del modello su un contesto differente.

Nel caso del dataset MERL, è stata selezionata la configurazione MERL2, che ha ottenuto risultati migliori rispetto all'altra variante. Questo modello ha raggiunto un'accuratezza dell'88%, mostrando buone prestazioni soprattutto nelle classi *Reach To Shelf* e *Retract From Shelf*. L'introduzione di tecniche di regolarizzazione, come il weight decay, ha permesso di ridurre l'overfitting, migliorando la stabilità del training e la capacità del modello di generalizzare su classi visivamente simili.

Per quanto riguarda il dataset ExtraEye, sono stati effettuati due differenti approcci di addestramento: uno basato sulla cross-validation e uno con data augmentation al 50%. In conclusione, il modello addestrato con tecniche di cross-validation ha ottenuto risultati mediamente discreti. Tuttavia, confrontando le medie delle metriche di performance, il modello basato su data augmentation ha mostrato prestazioni superiori.

Tra le diverse configurazioni, quella che ha ottenuto i risultati peggiori è la TPV, in quanto non è riuscita a raggiungere una loss ottimale, pur mostrando un leggero segnale di discesa. Tale andamento si è verificato sia nella versione con data augmentation che in quella con cross-validation, probabilmente a causa del numero limitato di epoche di addestramento o del fatto che non siano stati sbloccati a sufficienza i layer del modello durante il fine-tuning.

Infine, è stata condotta un'analisi comparativa tra le classi dei due dataset, evidenziando una forte somiglianza gestuale tra le classi *Inspect Product* (MERL) e *take* (ExtraEye). Tale sovrapposizione potrebbe causare ambiguità nel caso in cui i due dataset venissero utilizzati insieme in un unico processo di addestramento.

Nel complesso, il modello VideoMamba ha dimostrato ottime capacità di apprendimento e adattamento a contesti diversi, anche in presenza di dataset limitati e punti di vista multipli.